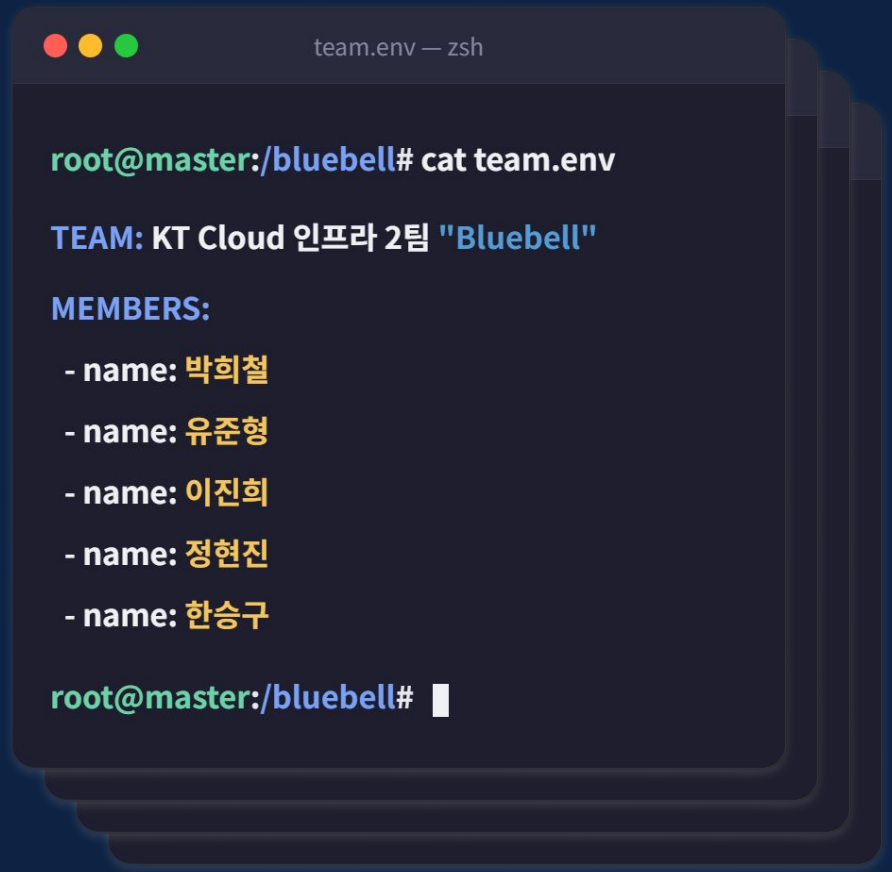
Bluebell

AWS + On-Premise 하이브리드 환경 3-Tier 자동화 및 모니터링 구축

구축 · 배포 · 모니터링 · 장애복구 자동화



Build

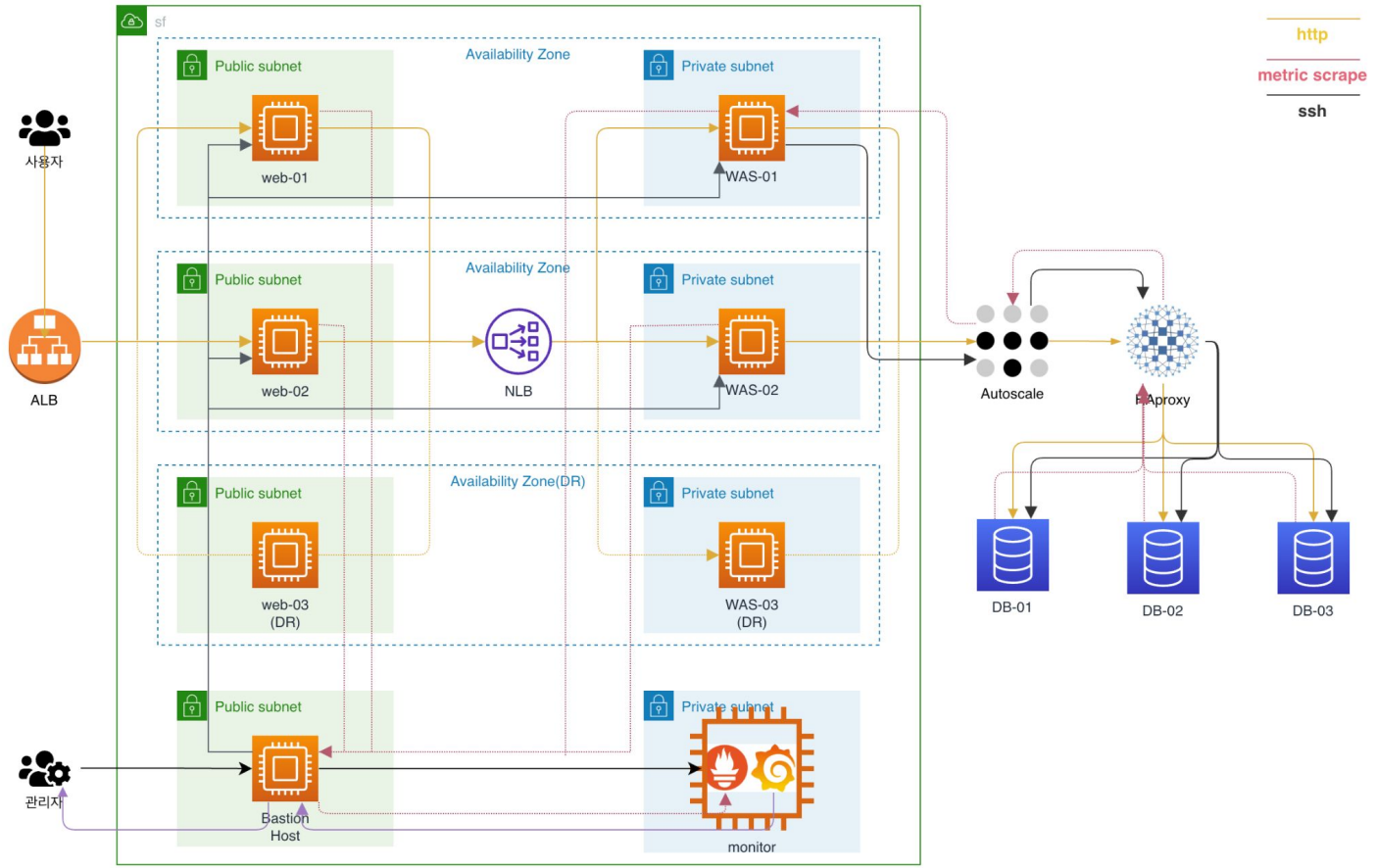
Deploy

Monitor

Recover

전체 Architecture

우리 Bluebell 팀이 구축한 전체 Architecture 구성입니다



사용자 접근 Flow

User → Public ALB → Web Nginx → NLB → WAS Flask API → HAproxy → MariaDB

관리 및 자동화 Flow

Bastion → Ansible → Docker Swarm → Recovery Script → ASG / Target Group

Metric 수집 Flow

Prometheus → node_exporter / cAdvisor / mysqld_exporter → Grafana

목차 및 Bluebell 프로젝트 한눈에 보기

단순 구축이 아니라 장애 이후 정상 상태로 되돌리는 운영 흐름을 구현했습니다

01

문제 인식

장애 대응 수작업
설정 차이 발생

02

구축

AWS Web-WAS-DB
2AZ·LB·TG

03

배포

Ansible
Docker Swarm

04

복구

EventBridge·SSM
ASG Replacement

05

검증

ALB·TG·Promethe
us

06

모니터링

Prometheus
Grafana Alerting

핵심 목표 장애가 발생해도 사람이 매달리지 않고, 시스템이 스스로 복구·검증까지 끝내는 인프라

문제 인식과 목표

장애가 발생해도 복구 절차가 반복 가능해야 운영형 인프라라고 볼 수 있습니다

문제 인식

- 장애 대응이 수작업에 의존
- 서버 재구축 시 설정 차이 발생
- 복구 결과를 검증하는 기준 부족
- 모니터링과 장애 대응 절차가 분리

Bluebell 목표

- 계층별 장애 구간 분리
- 자동화된 재구축과 서비스 배포
- 복구 후 정상 상태 자동 검증
- 모니터링으로 운영 상태 확인

네트워크와 보안 구조

외부 진입점과 내부 통신 경로를 분리했습니다

Public 영역

Internet 진입은 Public ALB 중심
Web 계층은 사용자 요청 수신

Private 영역

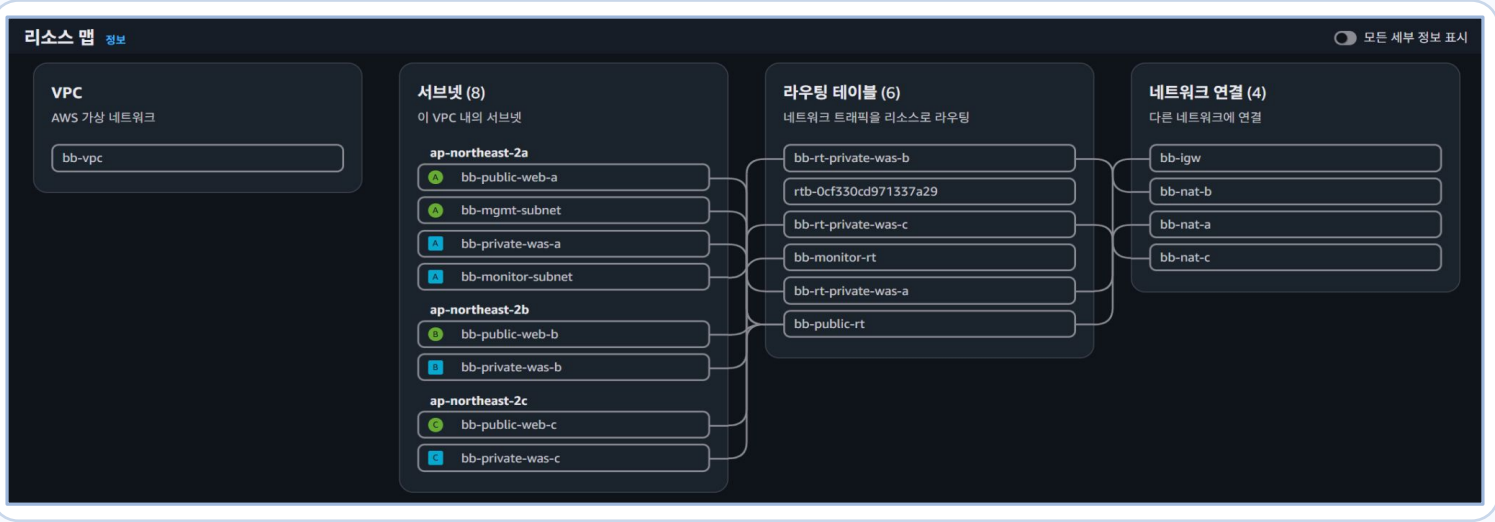
WAS와 DB는 직접 외부 노출 금지
Internal LB와 DB Proxy 중심 연결

관리 영역

Bastion은 Ansible·Swarm·복구 스크립트 실행 지점
운영자 관리 경로와 서비스 경로 분리

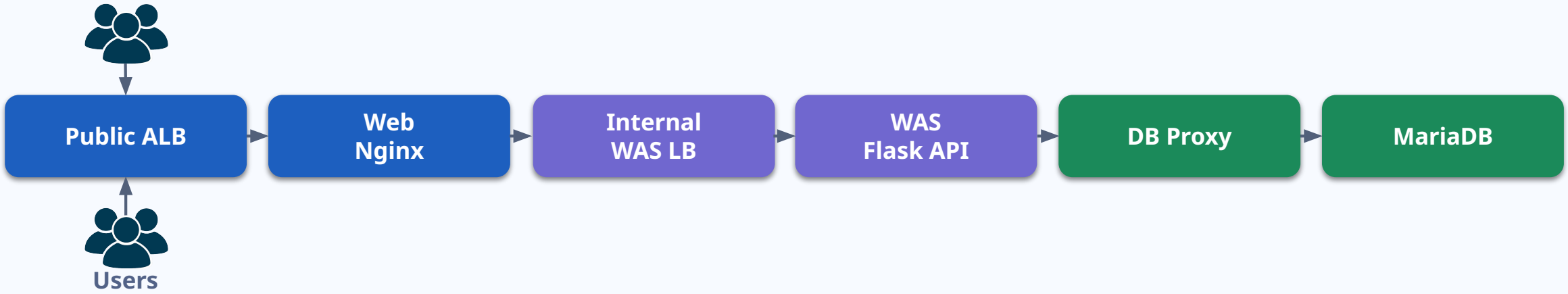
외부 패키지 경로

Private 서버의 패키지 설치와 이미지 pull은 NAT Gateway 경로 사용



Web-WAS-DB 요청 흐름

역할을 분리하면 보안, 확장, 장애 분석이 쉬워집니다



Web 정적 화면 제공
API 요청 프록시

WAS 비즈니스 로직
DB 접근 처리

DB 데이터 저장
WAS에서만 접근

검증 API: /api/health · /api/server · DB API

팀 구성과 담당 영역

각자 담당 영역을 나누고 최종적으로 하나의 운영 흐름으로 통합했습니다

담당자	주요 역할
박희철[팀장]	일정·문서·발표, **Web-WAS** 구축 및 **AWS/**통합 검증
정현진	**Ansible, Docker Swarm, IaC** 기반 자동화
한승구	**Web/Nginx** 구축
이진희	**Local** 환경에서의 **HA Proxy, DB** 및 백업 구성 구축
유준형	**Monitoring** 구축 자동화, **Monitoring** 스택 및 **Alerting** 프로비저닝 자동화 구축

통합 관점

팀원별 구현 결과를 Web → WAS → DB, 자동화, 모니터링 흐름으로 연결해 최종 발표와 시연영상으로 정리했습니다.

수행 절차

기본 연결 검증에서 시작해 이중화, 자동화, 장애복구, 모니터링으로 확장했습니다

1	2	3	4	5	6
기본 인프라	계층 연결	가용성 보완	자동 배포	장애복구	운영 검증
VPC/Subnet /EC2/SG	Web→WAS→DB	2AZ·ALB·TG	Ansible·Swarm	EventBridge· SSM·ASG	Monitoring· Baseline

핵심 진행 흐름

기본 인프라 구축 → Web/WAS/DB 연결 → 2AZ·LB 가용성 보완 → Ansible·Swarm 배포 자동화

→ EventBridge·SSM·ASG 장애복구 → Prometheus·Grafana 운영 검증

관리 및 협업

Notion 회의록/작업일지 , GitHub 브랜치/커밋. 멘트링 피드백을 장애복구 중심으로 반영

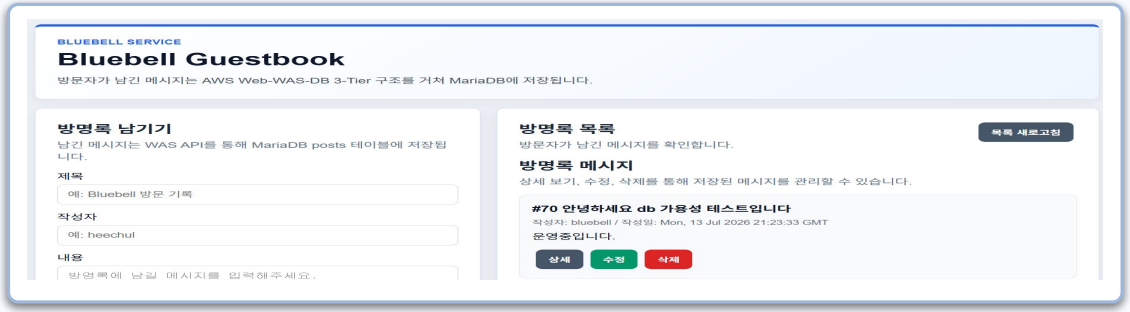
Web/WAS 서비스 구현 결과

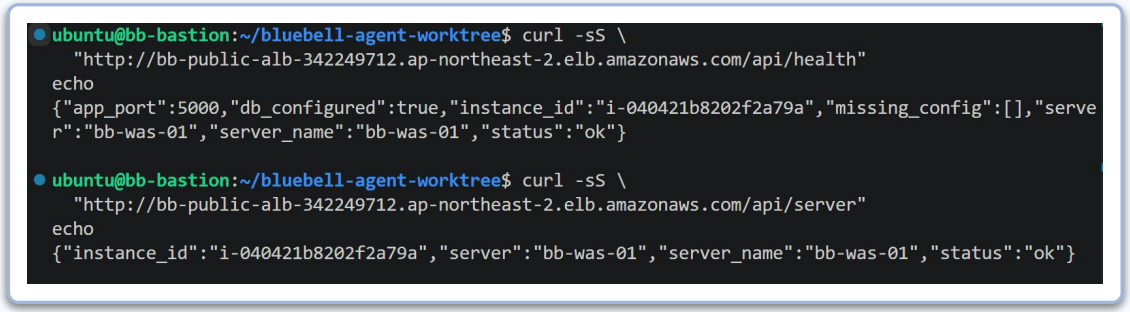
Nginx와 Flask API를 Docker Swarm 서비스로 배포하고 2 replica 기준으로 운영했습니다

- Web: Nginx 기반 정적 페이지 및 reverse proxy
- WAS: Flask API 기반 /api/health, /api/server 제공
- bluebell-web_nginx 2/2 · bluebell-was_flask 2/2
- Public ALB에서 Web → Internal WAS LB → WAS 흐름 검증

검증 기준

1. API 응답 성공
2. Service replica 2/2
3. Target Group healthy







Ansible과 Docker Swarm 자동화

수작업 명령을 playbook과 service 정의로 표준화해 반복 가능한 배포 구조를 만들었습니다

Ansible 자동화

- 01-bootstrap.yml: OS/Docker 준비
- 02-swarm.yml: worker join/label
- deploy-was.yml: WAS stack 배포
- deploy-web.yml: Web stack 배포
- exporter playbook: node_exporter/cAdvisor 배포

Docker Swarm 역할

- Bastion: Swarm manager
- Web/WAS: worker
- service desired state: 2 replica
- node label 기반 배치
- replacement node join/label

```
ubuntu@bb-bastion:~/bluebell-agent-worktree$ docker node ls
ID                            HOSTNAME              STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
2nwk6hqyf5gwve232ex5iu8xm *   bb-bastion            Ready     Active         Leader           29.1.3
up3zrqcpzj5l8rbs106pw04e3     i-02a461d9b1cd02299   Ready     Active                          29.6.1
l5feg7ozsdfrfamxhu67kpwzs     i-089e3a8d8d25ee5fe   Ready     Active                          29.6.1
os9dwsdiui97l4rchwfvu0of0     i-040421b8202f2a79a   Ready     Active                          29.6.1
nljviaunnl1f0nx0yfprzuzqg     i-06418176338740ab9   Ready     Active                          29.6.1
ubuntu@bb-bastion:~/bluebell-agent-worktree$ docker service ls | grep -E 'bluebell-web_nginx|bluebell-was_flask'
pv5u4icr4azy   bluebell-was_flask   replicated   2/2                 bluebell-was-flask:local   *:5000->5000/tcp
9qrn2y9epf1o   bluebell-web_nginx   replicated   2/2 (max 1 per node)   bluebell-web-nginx:local   *:80->80/tcp
```

장애복구 자동화가 필요한 이유

Swarm은 컨테이너 장애를 다루지만 EC2 자체 장애에는 인프라 복구 절차가 필요했습니다

Swarm이 처리하기 쉬운 장애

- 컨테이너 프로세스 종료
- Task 실패
- Replica 부족
- 살아 있는 worker 간 재배치

추가 자동화가 필요한 장애

- EC2 stopped/terminated
- Worker node 소멸
- Target Group stale target 잔존
- Replacement 서버 Swarm 미가입
- Node label 누락

해결 방향

EventBridge 감지 → SSM 실행 → ASG replacement → Swarm join/label → stale 정리 → service 2/2 converge

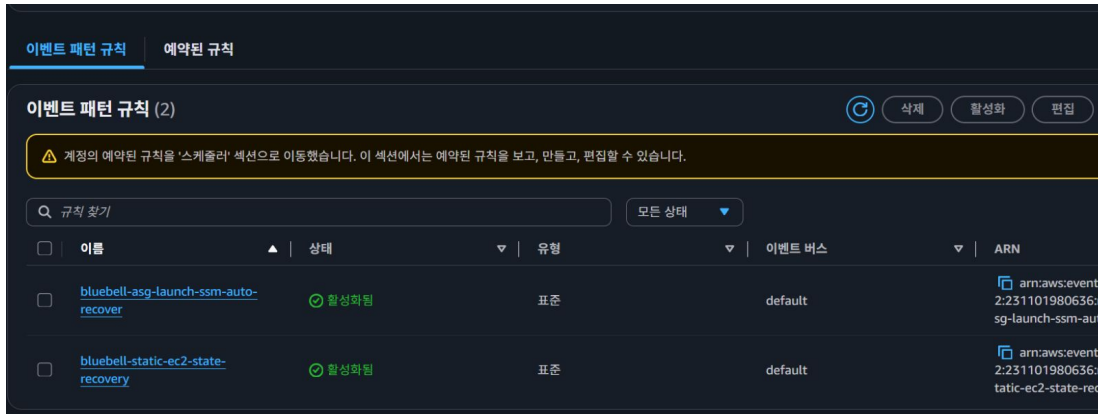
EventBridge·SSM·ASG 기반 자동복구 구조

인스턴스 상태 변경 이벤트를 Bastion 복구 스크립트로 연결해 복구 절차를 실행합니다

EC2 상태 이벤트 → EventBridge → SSM Run Command → Bastion 복구 스크립트 → ASG / Swarm / TG 수렴 → Validation

- stopping/stopped/shutting-down/terminated 이벤트 감지
- InputTransformer로 instance-id/state 전달
- EventBridge Rule Enabled 상태에서 실제 Web 장애 감지

- bluebell-replacement-recover.sh --repair 실행
- ASG Replacement 생성 → Swarm join/label
- Target Group 수렴 후 service 2/2 검증





ASG Replacement 실제 검증 결과

신규 Web 노드 생성부터 서비스 편입, Target Group, Grafana 관측 연속성까지 검증했습니다

RECOVERY FLOW VERIFIED · SERVICE / MONITORING CONTINUITY

Replacement

시연 시 신규 **Web** 노드 생성
Private IP 자동 할당

Inventory / Bootstrap

web · swarm_workers
Ansible failed=0

Swarm / Service

role=web · task Running
Web/WAS 2/2 유지

ALB / TG

TG healthy
/ · /api/health · /api/server

Monitoring

node_exporter · cAdvisor
Grafana 관측 대상 갱신

Alerting

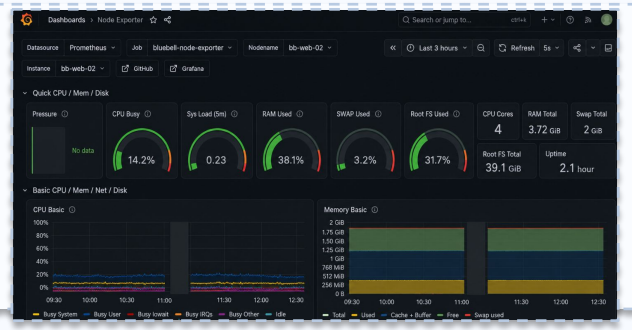
Instance Down 감지
Email/Contact Point 알림

```
===== ASG REPLACEMENT =====
INSTANCE_ID : i-02e8afd0c3bd8144b
PRIVATE_IP  : 10.0.1.207

===== DYNAMIC INVENTORY =====
[web]
  i-02e8afd0c3bd8144b
[swarm_workers]
  i-02e8afd0c3bd8144b
```

```
===== REPLACEMENT =====
INSTANCE_ID : i-02e8afd0c3bd8144b
PRIVATE_IP  : 10.0.1.207

===== TARGET GROUP =====
TARGET_GROUP : bb-recovery-web-tg
-------------------------------------------------------
|              DescribeTargetHealth                   |
+----------------------+-------+---------+-----------+
|      Instance        | Port  | Reason  |   State   |
+----------------------+-------+---------+-----------+
|  i-02e8afd0c3bd8144b |  80   |  None   |  healthy  |
+----------------------+-------+---------+-----------+
```



DB 구성

DB서버 3대를 클러스터로 구성하였고, 앞에 proxy서버를 두어 부하분산을 하였습니다

비동기 데이터 복제

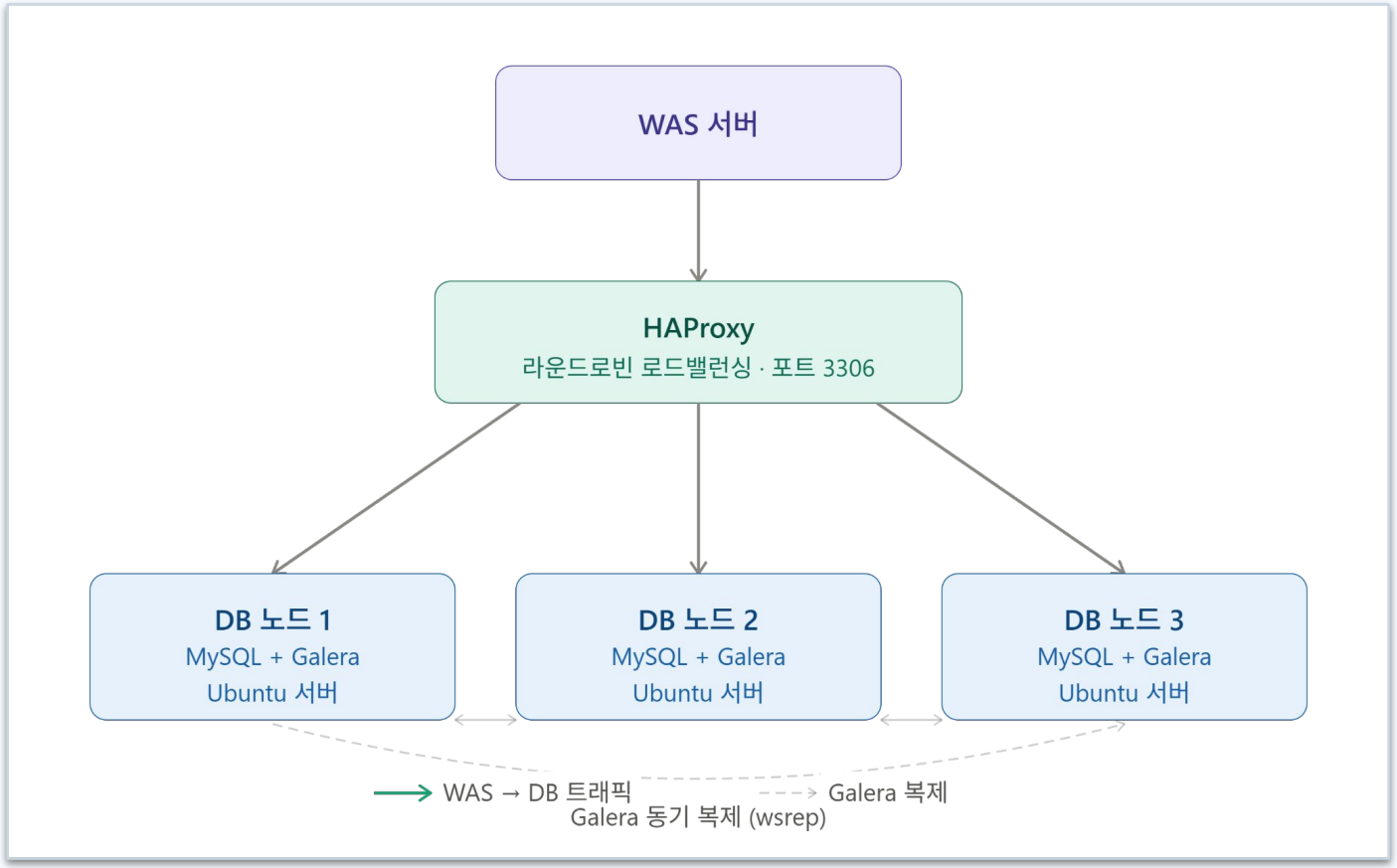
노드 간 실시간 비동기 동기화로 성능 최적화

고가용성(HA) 보장

1개 노드 장애(Down) 시에도 2개 노드로 무중단 서비스 지속

자동 복구 및 정합성

장애 노드 복구 시 재동기화 (Replication)를 통한 데이터 일치



DB Backup과 Secret 관리

dbdump를 다른 서버로 백업하고, 생성된 백업 파일을 AWS S3에 업로드합니다

백업 자동화 (Cron)

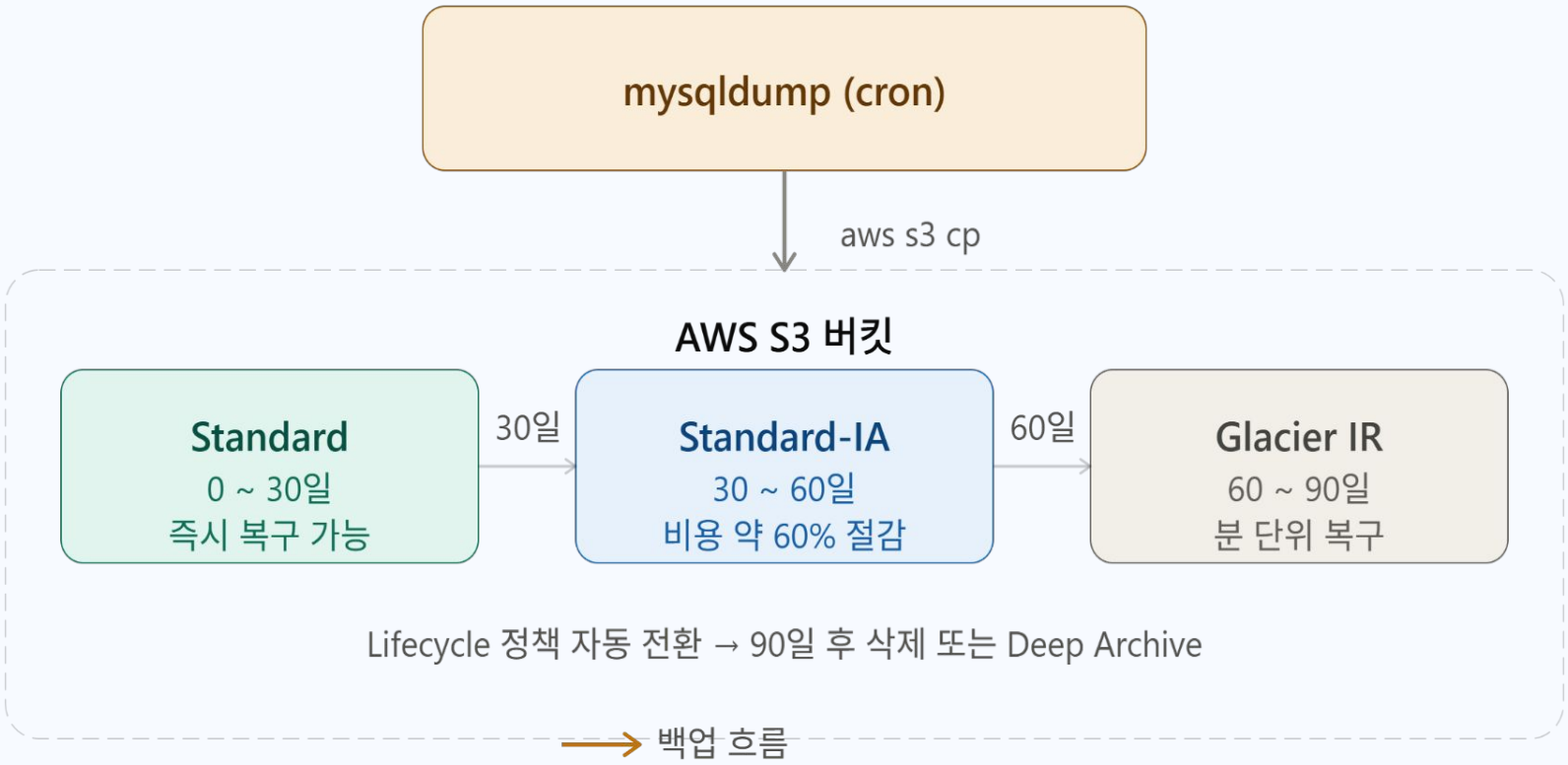
매일 새벽 2시 정기 DB 덤프(Dump) 스크립트 자동 실행

클라우드 이관 (AWS S3)

생성된 백업 파일을 AWS S3 스토리지로 안전하게 업로드

비용 최적화 (Lifecycle)

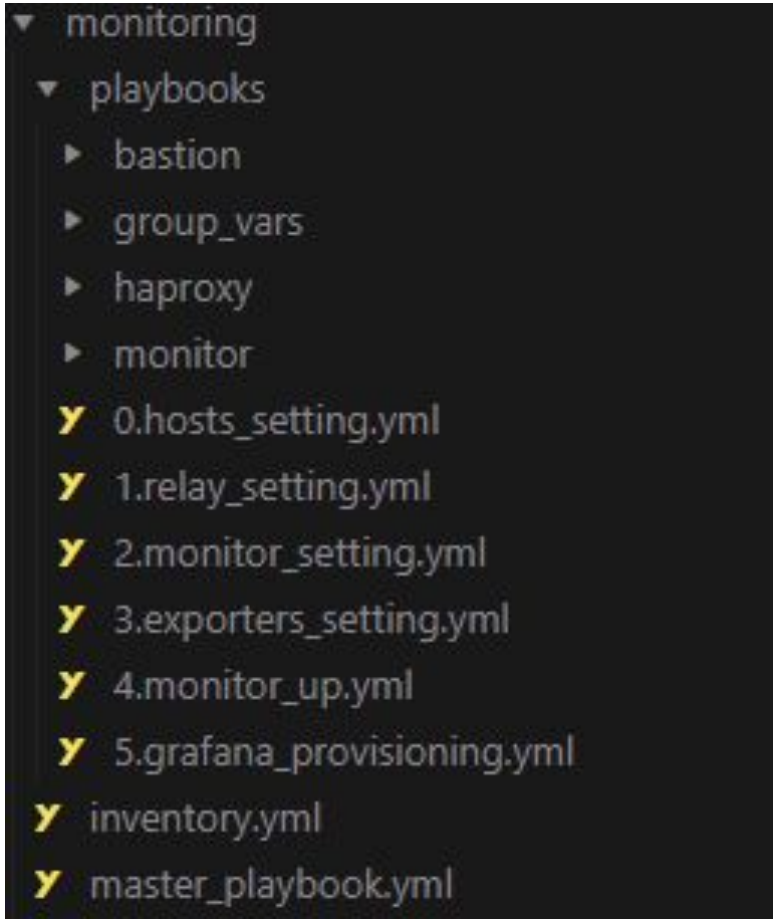
S3 수명주기 정책(Lifecycle) 적용을 통한 스토리지 비용 절감



Prometheus/Grafana 모니터링 구축

Bastion을 통해 인터넷이 격리된 Monitor 인스턴스에 모니터링 스택을 배포하고, 전 노드에 Exporter들을 자동 배포했습니다

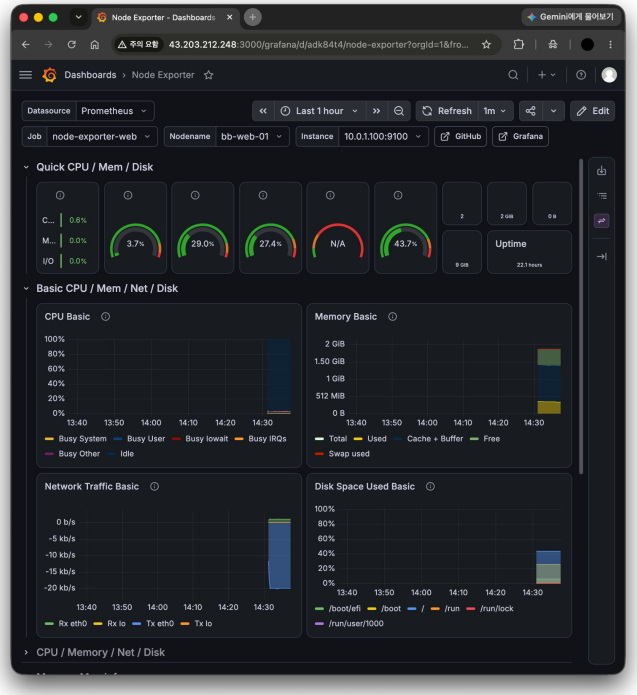
0. hosts_setting	known_hosts 정리, inventory 기재 노드 hostname 설정, 접속용 공개키 등록
1. relay_setting	로컬 VM DB metric 접근을 위해 WAS socat relay와 HAProxy TCP 포워딩 규칙 배포
2. monitor_setting	monitor 인스턴스에 docker, docker compose, monitoring stack 파일 전달 후 docker 설치·기동 및 이미지 로드
3. exporter_setting	각 노드에 node exp, cAdvisor, mysqld exp 자동 배포
4. monitor_up	prometheus.yml과 docker-compose.yml 전송 후 Grafana와 Prometheus 기동
5. grafana_provisioning	DR 대비 Dashboard, Alerting 설정을 그대로 프로비저닝



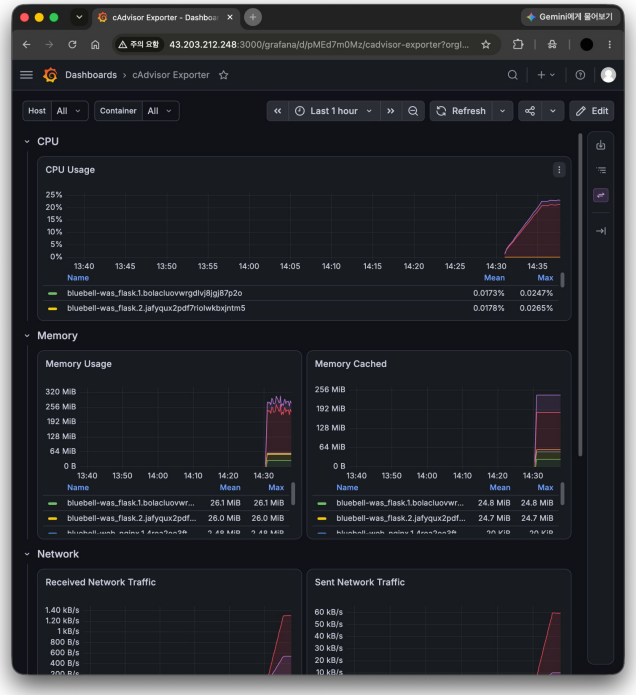
Prometheus/Grafana 모니터링 Dashboards

운영 상태를 눈으로 확인하기 위해 서버와 컨테이너 메트릭을 수집하고 시각화했습니다

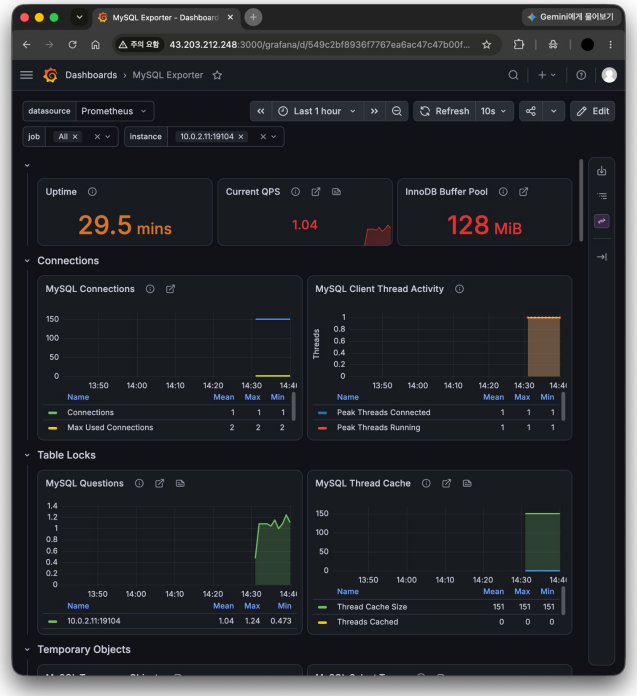
Node exporter Dashboard
전 Node 대상



cAdvisor Dashboard
Web, WAS 그룹 대상



Mysqld exporter Dashboard
DB 그룹 대상



Metric ports node_exporter(9100), cAdvisor(8080, 8081 for WAS), mysqld_exporter(9104)

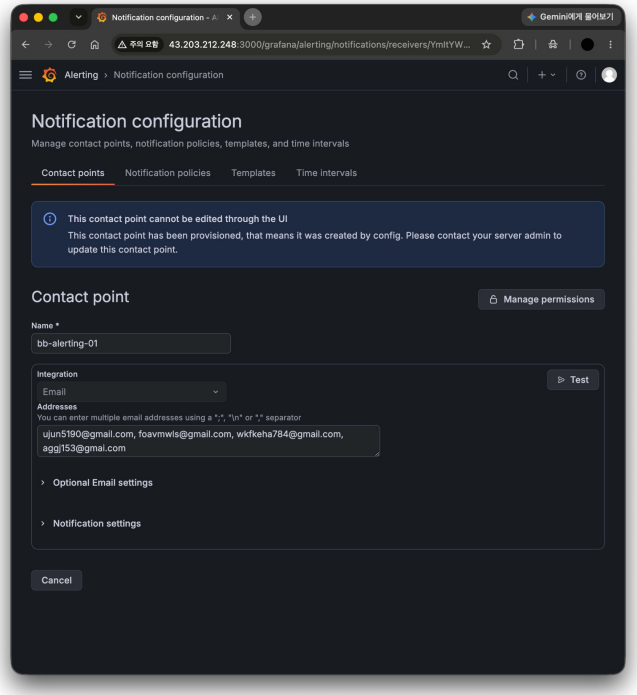
Prometheus/Grafana 모니터링 Alerting

Bluebell

운영 시 발생될 수 있는 장애 상황에 담당자가 더욱 빠르게 대처할 수 있게 Contact point와 Alerting rule을 설정했습니다

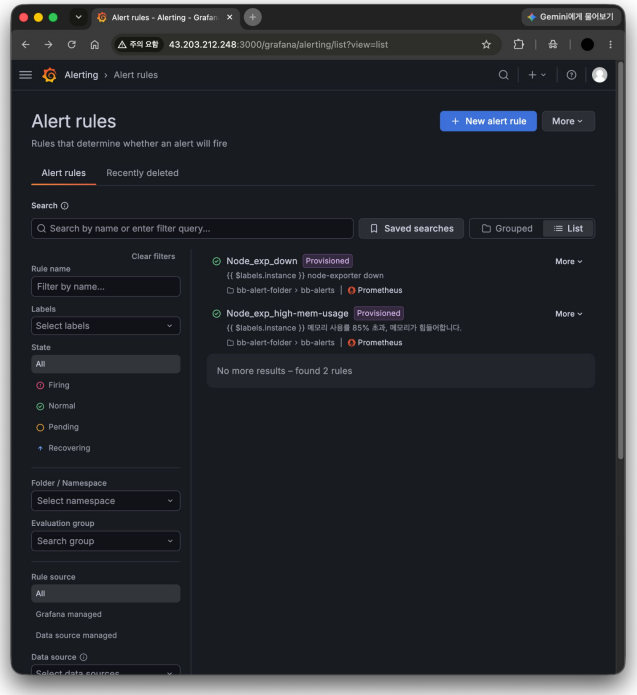
Contact Point

gmail smtp 서버를 이용한 mailing



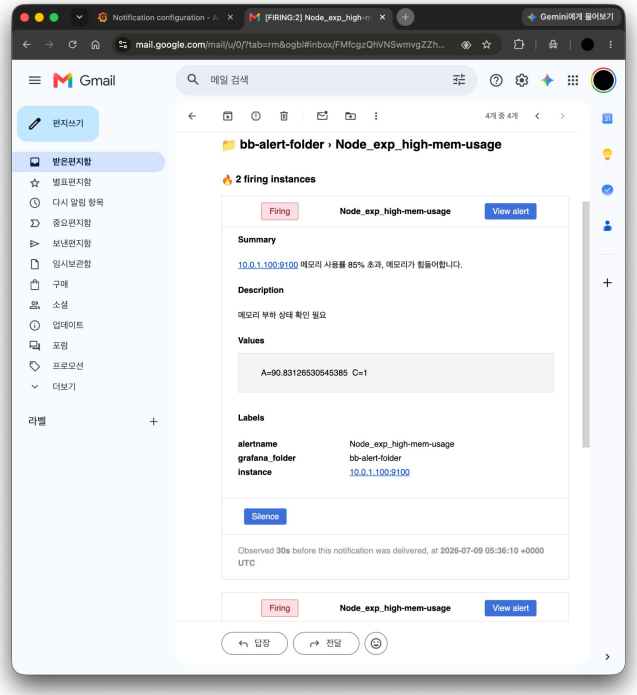
Alerting Rules

Node down, Memory high 상황 시 Notify



Alerting Results

Stress test로 이슈를 발생시켜 전달된 Alert



최종 구현 및 검증 현황

최종 E2E 검증 결과를 기준으로 구현 완료와 후속 개선 영역을 구분했습니다

영역	상태	비고
AWS 네트워크 /LB	구현·검증 완료	Public/Internal LB, TG
Web/WAS API	구현·검증 완료	/api/health, /api/server
Ansible/Swarm	구현·검증 완료	Web/WAS 2/2
ASG Replacement	구현·검증 완료	신규 노드 join/label/exporter 및 관측 연속성
DB	구현 완료	3노드, Proxy, 백업 /S3
Monitoring	구현·검증 완료	Dashboard/Alerting/Replacement 관측 연속성
EventBridge/SSM	구성·검증 완료	Rule Enabled 기준 Web 장애 감지 복구 검증

결론과 개선 방향

운영 자동화 흐름을 검증했고, 실제 운영 수준으로 확장하기 위한 보완 과제를 정리했습니다

잘한 점

- Web-WAS-DB 계층 분리
- Ansible + Swarm 반복 배포
- EventBridge/SSM/ASG 장애복구
- Grafana Alerting· Contact Point
- Replacement 관측 연속성 검증

아쉬운 점

- 반복 장애 테스트 시간 제한
- DB 정합성/failover 고도화 필요
- 보안그룹 최소화와 HTTPS 보완 필요
- 알림 체계 이중화 보완 필요

개선 방향

- RTO/RPO 측정 자동화
- HTTPS/ACM/WAF 적용
- DB Multi-AZ 또는 관리형 DB 검토
- IaC(Terraform/CloudFormation) 확장
- Secret rotation · 알림 정책 고도화

Q&A

이상 **Bluebell** 팀의 프로젝트 발표였습니다.
경청해주셔서 감사합니다.